Real-world (a cold interview on your own project) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — The idea, and the four beats
- Hint 2 — The plan, and almost the whole walkthrough

Hint 1 — The idea, and the four beats

A project walkthrough is a different skill than answering a factual question — it's **narrative construction under the assumption of zero shared context**. The interviewer has never seen your README, your CURRICULUM.md, or your code. Every acronym or internal name (Doc11, "supervisor pattern") needs either a one-clause explanation or needs to be avoided in favor of a plain description.

Use Project 4 (ContentForge) as this hint's working example. Structure it the way this document's own Four-Depth Format structures any answer — start at the 30-second claim ("what it does, for whom"), then let it rise into Normal depth: **problem → architecture at a glance → one decision worth defending → one honest limitation.** Interviewers specifically listen for whether you can identify your own project's weakest point unprompted — volunteering it (briefly, not apologetically) is a stronger signal than waiting to be caught.

Draft your own four-part outline (for any project of your choice) before Hint 2.

When the interviewer pushes back:

Two curveballs happen often in real project-walkthrough interviews, and both are worth rehearsing separately from the plain four-beat walkthrough:

Curveball 1 — the direct challenge. Right after you name "one decision worth defending," a skeptical interviewer pushes back with a simpler alternative: *"Why not just use a fixed pipeline with one retry loop instead of a full supervisor — isn't the supervisor added complexity for one edge case?"* The wrong move is either caving instantly ("yeah, maybe that would've been simpler") or getting defensive and re-explaining the same decision louder. The right move names the *specific requirement* the simpler alternative can't meet, concedes the real cost of your choice honestly, and — if the challenge has a genuine point — says so.

Curveball 2 — the cold project switch. Mid-conversation, the interviewer says: *"Interesting — tell me about a different project instead, one where something went wrong."* There's no prep time for this. The skill is having the same four-beat shape (problem → architecture → decision → limitation) memorized as a *reusable structure*, not memorized *per project* — so you can slot any project into it on the spot, including one you didn't expect to discuss.

The real design question isn't "do I know my project well" — it's "can I defend one real decision under a direct, reasonable challenge, and can I produce the same four-beat shape for a project I didn't prepare tonight?"

Try both before Hint 2: defend the supervisor-vs-fixed-pipeline decision under pushback, and outline a different project's four beats from memory, cold.

Hint 2 — The plan, and almost the whole walkthrough

The same plan, at the level you'd deliver it live:

```
problem: state the user-facing need, not the technical implementation

shape: name the pattern (Doc11's supervisor pattern), name the agents,
       and connect the pattern choice to a SPECIFIC requirement
       ("needs to loop back" -> "supervisor can route backward, a fixed
       pipeline can't")

one decision: pick something with a real trade-off — e.g. "why a
              supervisor and not a fixed sequential pipeline" — and be
              ready to say what you gave up (a supervisor is harder to
              reason about statically than a fixed pipeline)

one limitation: name a genuine gap (a stage that always runs even when
                it isn't needed, no cost cap per run) and the specific
                fix you'd add, not just "it's not perfect"
```

Notice the "one decision" section explicitly includes what you *gave up* — a decision defended without acknowledging its cost sounds like marketing, not engineering judgment.

Fill this in for your own project, say it out loud once, then practice the curveballs below.

The curveballs:

The plan for defending the decision under direct challenge:

```
your decision, as stated: "supervisor instead of a fixed pipeline, because
                           rejected drafts need to route backward"
```

```
interviewer challenge: "isn't that added complexity for one edge case? a
                        fixed pipeline with a single retry loop around the
                        review step would handle that too, and it's
                        simpler to reason about."

your move:
    concede what's true: a single conditional retry loop IS simpler, and
        for exactly one back-edge, it might genuinely be enough
    name the specific thing it stops covering:
        a fixed retry loop only knows how to go back to ONE fixed place;
        if a future specialist (e.g. an SEO-check step) also needs to be
        able to send work backward, the fixed pipeline needs a new
        hard-coded loop for every new case — the supervisor already
        generalizes to that without new wiring
    name what you gave up honestly:
        harder to statically trace than either option, more moving parts
        to test
    land on the actual call:
        for the CURRENT scope, a fixed retry loop might have been enough
        — the supervisor was a bet on the pipeline growing more
        conditional paths, which is a judgment call, not an obvious win
```

And the plan for the cold project-switch, as a reusable shape (not memorized per project):

```
for ANY project, on demand:
    problem -> one sentence, user-facing, no jargon
    shape -> pattern name + agents/parts + why that pattern fit
    one decision -> something debatable, plus what you gave up
    one limitation -> a real gap, plus the fix you'd add

practice this shape against a SECOND project you didn't plan to discuss
tonight, cold, before checking the Solution
```

Full solution: Show me the solution